Gen AI 시대의 파이썬: 코딩이 아닌 설계를 하라

AI를 나만의 '수석 개발자'로 채용하는 법

이동현

2026년 2월 11일

한국외국어대학교 Social Science & AI융합학부

세션 시간표

세션	시간	주요 내용
1	09:30 – 10:20	세션 1 복습, 오늘 실습 환경 점검
2	10:30 – 11:20	함수 기초: 정의, 인자, 반환값
3	11:30 – 12:20	함수 심화: 스코프, 람다 / **클래스(OOP) 기초**
4	12:30 – 13:20	모듈과 패키지, 라이브러리 활용, 구조 설계
	13:20 – 14:20	**점심 시간**
5	14:20 – 15:10	예외 처리: `try-except-finally` 패턴
6	15:20 – 16:10	RSS와 XML 구조 이해, `feedparser` 소개
7	16:20 – 17:10	RSS 데이터 파싱 및 필터링 로직 설계
8	17:20 – 18:10	RSS 미니 프로젝트 통합 및 회고, Q&A

세션 1 복습

세션 1 복습: 조건문 & 반복문

- **목표**
  - 조건문(`if, elif, else`)과 반복문(`for, while`) 문법을 다시 확인합니다.
  - 중첩 반복, 나눗셈/나머지 연산을 활용하여 **구구단, 공약수, 소수** 문제를 풀어 봅니다.
- **연습 문제 유형**
  - 1) 전체 구구단 출력
  - 2) 두 수의 공약수 / 최대공약수 / 최소공배수
  - 3) 주어진 범위의 소수 찾기

세션 1 복습: 연습 문제

문제 1 – 전체 구구단 출력
1단부터 9단까지, 각 단의 곱셈식과 결과를 모두 출력하는 프로그램을 작성하세요.

문제 2 – 공약수, 최대공약수, 최소공배수
양의 정수 두 개가 주어졌을 때,

- (a) 두 수의 공약수 목록
- (b) 두 수의 최대공약수(GCD)
- (c) 두 수의 최소공배수(LCM)를 구하는 프로그램을 작성하세요.

문제 3 – 소수 찾기
2 이상 정수 n이 주어졌을 때, 2부터 n까지의 소수를 모두 출력하는 프로그램을 작성하세요.

구구단 풀이 코드 & 프롬프트 예시

풀이 코드 (예시)

```
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{i} x {j} = {i * j}")
```

AI에게 시킬 때 사용할 프롬프트 (정석 예시)

> ”파이썬으로 *1*단부터 *9*단까지 전체 구구단을 출력하는 코드를 작성해줘. 중첩 *for* 반복문을 사용하고, 각 줄에 ‘*a x b = c*‘ 형태로 출력해줘. *PEP 8* 스타일을 최대한 지켜줘.”

공약수 / GCD / LCM 풀이 코드 & 프롬프트

풀이 코드 (예시)

```python
a = 12
b = 18

common_divisors = []

for d in range(1, min(a, b) + 1):
    if a % d == 0 and b % d == 0:
        common_divisors.append(d)

gcd = common_divisors[-1]
lcm = a * b // gcd

print("Common divisors:", common_divisors)
print("GCD:", gcd)
print("LCM:", lcm)
```

공약수 / GCD / LCM 프롬프트 예시

AI에게 시킬 때 사용할 프롬프트 (정석 예시)

> ”파이썬으로 양의 정수 두 개를 입력으로 받아 *1*) 두 수의 공약수 리스트, *2*) 최대공약수(*GCD*), *3*) 최소공배수(*LCM*)를 계산하는 함수를 작성해줘. 유클리드 호제법을 사용해서 *GCD*를 구하고, 타입 힌트와 간단한 *docstring*도 포함해줘.”

소수 찾기 풀이 코드 & 프롬프트

풀이 코드 (예시)

```python
limit = 50

for n in range(2, limit + 1):
    is_prime = True
    for d in range(2, int(n**0.5) + 1):
        if n % d == 0:
            is_prime = False
            break
    if is_prime:
        print(n)
```

AI에게 시킬 때 사용할 프롬프트 (정석 예시)

"파이썬으로 2부터 *n*까지의 소수를 모두 찾아 리스트로 반환하는 함수를 작성해줘. 시간 복잡도를 고려해서, *n*까지 모든 수로 나누지 말고 제곱근까지만 나누는 방식으로 구현해줘. 이름이 *is_prime*인 보조 함수와

오늘의 목표: "개발자"에서 "아키텍트"로

- **과거의 코딩**: 문법 암기 → 한 줄씩 작성 → 에러 수정
- **AI 시대의 코딩**: 시스템 설계 → AI에게 지시(Prompt) → 조립 및 검증
- **핵심 역량**:
  1. **구조화 (Structure)**: 무엇을 만들지 그림 그리기
  2. **지시 (Prompting)**: AI가 이해하는 언어로 업무 위임하기
  3. **검증 (Verification)**: AI의 결과물이 의도와 맞는지 확인하기

학습 로드맵 (09:30 ~ 18:10)

- **오전: AI와 소통하는 법 (기초)**
  - 1세션: 셋업 & 마인드셋 - "AI는 내 파트너"
  - 2세션: 함수(Function) - "구체적으로 일 시키기"
  - 3세션: 구조화(Structure) - "레고 블록 조립하기"
  - 4세션: 모듈과 패키지 - "도구함 정리하기"
- **오후: AI로 시스템 구축하기 (실전)**
  - 5세션: 디버깅 — "AI와 함께 문제 해결하기"
  - 6–8세션: [미니 프로젝트] 나만의 RSS 뉴스 브리핑 봇 만들기

Python 기초 리마인드

Python 소개 & 개발 환경

- Python: 귀도 반 로섬이 1991년에 개발한 인터프리터 언어
- **장점**
  - 간결한 문법, 오픈 소프트웨어
  - 풍부한 생태계: 데이터 분석, 머신러닝, 웹 개발 등
- **개발 환경**
  - Anaconda 배포판 + Jupyter Notebook
  - VS Code / PyCharm / Antigravity IDE 등

프로그래밍 기초 & Jupyter Notebook

- **프로그래밍**: 컴퓨터가 수행할 명령어(알고리즘)를 코드로 표현
- **좋은 코드의 조건**
  - 명확한 **명세**와 **주석**
  - 일관된 **들여쓰기** (space 4개 권장)
- Jupyter Notebook **핵심**
  - 셀 단위 실행: `Shift+Enter`, `Ctrl+Enter`
  - Command / Edit 모드와 기본 단축키 (a, b, dd, x, c, v, Shift+m 등)

자료형(1) – 단일 자료형

- **변수**: 데이터를 저장하는 이름 있는 공간
- **주요 단일 자료형**
  - `int`: 정수형
  - `float`: 실수형
  - `complex`: 복소수형
  - `bool`: 논리형 (`True` / `False`)
- **내장 함수**
  - `type(x)`, `len(x)`, `print(x)`

자료형(2) – 군집 자료형

- **문자열** (`str`)
  - 인덱싱/슬라이싱: `s[0]`, `s[1:4]`, `s[::-1]`
  - 포맷팅: `"hello %s" % name`, f-string
- **리스트** (`list`)
  - 가변 시퀀스: `append`, `insert`, `pop`
- **튜플** (`tuple`), **집합** (`set`), **딕셔너리** (`dict`)
  - 튜플: 변경 불가, 소괄호 사용
  - 집합: 중복 없는 원소 집합, 집합 연산 지원
  - 딕셔너리: `{key: value}` 구조, `keys()`, `values()`, `items()`

연산자 & 조건문/반복문 요약

- **연산자**
  - 산술: `+ - * / ** % //`
  - 비교: `>, >=, <, <=, ==, !=`
  - 논리: `and, or, not`, 멤버: `in, not in`
- **조건문**
  - `if elif else` 구조로 분기 제어
- **반복문**
  - `for`: 시퀀스 순회, `range()`와 함께 많이 사용
  - `while`: 조건이 참인 동안 반복, 무한 루프 주의
  - 제어: `break, continue, pass`

함수 & Pythonic Code

- **함수 기초**
  - 정의: `def func(a, b=0): return a + b`
  - 가변 인자: `*args`, `**kwargs`
  - 람다: `lambda x: x * 2`
- Pythonic Code
  - 리스트 컴프리헨션: `[i**2 for i in range(10) if i % 2 == 0]`
  - enumerate: `for i, v in enumerate(items): ...`

입출력, 클래스, 모듈/패키지, 예외 처리 한눈에 보기

- 입출력
  - `input()`, `open`, `read`, `write`, `with open(...)`
- 클래스
  - `class MyClass:` / `__init__`, 인스턴스 메서드, 상속
- 모듈/패키지
  - `import`, `from ... import ...`, `as` 별칭
  - `if __name__ == "__main__":` 패턴
- 예외 처리
  - `try-except-else-finally`, `raise`, `with` 문

패러다임의 변화: AI Partner

기존 방식 (Coder)

- 문법 책을 찾아본다.
- 오타 하나에 1시간 고민한다.
- 처음부터 끝까지 직접 짠다.

→

AI 방식 (Architect)

- **큰 그림**을 그린다.
- AI에게 **명확한 작업 지시서**를 준다.
- **코드 리뷰**를 하고 조립한다.

핵심
여러분은 단순 '코더'가 아니라 '시스템 설계자'입니다.

함수(Function) ≈ 업무 지시서

- 함수란? **입력(Input)**을 받아 **처리(Process)**하고 **출력(Output)**하는 도구
- AI에게 함수를 짜달라고 할 때 가장 중요한 3요소:
  1. **역할(Role)**: 이 함수가 할 일의 정의
  2. **입력(Input)**: 무엇을 받아야 하는가?
  3. **출력(Output)**: 결과로 무엇을 줘야 하는가?

함수의 해부학 1: 매개변수와 반환값

- **매개변수 (Parameter)**: 함수가 받는 재료
  - **위치 인자**: 순서대로 들어가는 인자 (`a, b`)
  - **기본값 인자**: 값이 없으면 기본값 사용 (`rate=0.1`)
- **반환값 (Return)**: 함수가 내뱉는 결과물
  - `return`이 없으면 None 반환
  - 여러 값을 튜플로 반환 가능 (`return sum, avg`)

```python
def calculate(a, b, op='+'):
    """Simple calculator with default operator."""
    if op == '+':
        return a + b
    return None
```

함수의 해부학 2: Docstring과 타입 힌트

Goal: AI와 동료(혹은 미래의 나)를 위한 친절한 설명서

- Docstring: 함수 바로 아래 `""" ... """`로 설명 작성
- Type Hint: 인자와 반환값의 타입을 명시 (`: int`, `-> bool`)

```python
def greet(name: str) -> str:
    """Return a welcome message for the given name."""
    return f"Hello, {name}!"
```

프롬프트 엔지니어링 실습: 함수 생성

[Bad Prompt]

"부가세 계산하는 코드 짜줘." (너무 모호함)

[Good Prompt: **구조화된 지시**]

파이썬 함수를 작성해줘.

1. 함수명: `calculate_vat`
2. 입력: `price` (금액, 정수), `rate` (비율, 기본값 *0.1*)
3. 기능: 가격에 부가세를 곱한 금액을 반환.
4. **문서화**: *Google Style Docstring*과 *Type Hint*를 꼭 포함해줘.

AI가 작성한 결과물 (예시)

```python
def calculate_vat(price: int, rate: float = 0.1) -> float:
    """Calculate VAT from price and rate."""
    if price < 0:
        raise ValueError("Price cannot be negative.")

    return price * rate
```

- **우리의 역할**: 코드를 한 줄씩 치는 게 아니라, **입력 조건**과 **예외 상황**을 정의해주는 것.

함수 심화 1: 유연한 함수 (*args, **kwargs)

Goal: 입력이 몇 개일지 모를 때 (예: 뉴스 기사 10개? 100개?)

- `*args`: 여러 개의 인자를 튜플(Tuple)로 받음
- `**kwargs`: 키워드 인자를 딕셔너리(Dict)로 받음

```
def summarize_news(*articles, **options):
    limit = options.get('limit', 5)
    for i, article in enumerate(articles[:limit]):
        print(f"{i+1}. {article}")
```

Prompt Hint: "The number of arguments can vary, so please use `*args` to make the function flexible."

함수 심화 2: 람다(Lambda)와 스코프(Scope)

Lambda (일회용 함수)

- 이름 없이 간단하게 정의
- `lambda x: x * 2`
- 주로 정렬(sort)이나 필터(filter)에 사용

Scope (변수의 범위)

- Local: 함수 안에서만 생존
- Global: 프로그램 전체 생존
- **주의**: 함수 안에서 밖에 있는 변수를 수정하려면 `global` 키워드 필요 (AI가 자주 실수함)

세션 3: 클래스와 객체 지향 (OOP)

클래스(Class)란? - 붕어빵 틀과 붕어빵

- **절차 지향 vs 객체 지향**
  - 절차 지향: 함수(기능) 위주로 코드 작성
  - 객체 지향: 데이터(변수)와 기능(함수)을 묶어서 '객체'로 관리
- **비유**
  - **클래스 (Class)**: 설계도, 붕어빵 틀 (`class Dog:`)
  - **인스턴스 (Instance)**: 실제 붕어빵, 강아지 (`my_dog = Dog()`)

클래스의 구조: __init__과 self

가장 헷갈리는 self: "나 자신"을 가리키는 말

```
class Dog:
    def __init__(self, name, age):
        # self.attribute = instance-specific state
        self.name = name
        self.age = age

    def bark(self):
        # access instance attributes via self
        print(f"{self.name} says Wal-Wal!")

d = Dog("Baduki", 3)
d.bark()  # Baduki says Wal-Wal!
```

상속(Inheritance): 부모의 능력을 물려받기

Goal: 코드를 복사-붙여넣기 하지 않고 재사용하기

- **부모 클래스**: Animal (공통 기능)
- **자식 클래스**: Dog, Cat (특화 기능)

```
class Animal:
    def eat(self):
        print("Yum yum")

class Cat(Animal):  # inherits from Animal
    def meow(self):
        print("Meow!")

c = Cat()
c.eat()   # use parent method
c.meow()  # use child method
```

클래스 설계를 위한 프롬프트

AI에게는 **"속성(변수)"**과 **"행동(메서드)"**을 정의해줘야 합니다.

[Prompt]

"파이썬 클래스 *BankAccount*를 만들어줘.

1. **속성**: 소유자 이름(*owner*), 잔액(*balance*, 비공개 변수로)
2. **생성자**: 이름과 초기 잔액을 받아서 초기화해.
3. **메서드**:
   - *deposit(amount)*: 입금
   - *withdraw(amount)*: 출금 (잔액 부족 시 에러 발생)

*Docstring*을 포함해서 작성해."

세션 4: 모듈과 패키지, 라이브러리

시스템 아키텍처: 레고 블록 조립

코드가 길어지면 **관리**가 안 됩니다. 서랍장에 정리하듯 코드를 나눠야 합니다.

폴더 구조 (Structure)

- `main.py`: 실행 본부
- `data_loader.py`: 재료 가져오기
- `processor.py`: 요리하기
- `saver.py`: 접시에 담기

프롬프트 예시

"뉴스 데이터를 수집해서 엑셀로 저장하는 프로그램을 만들고 싶어. 기능별로 파일을 어떻게 나누는 게 좋을지 **패키지 구조**를 제안해줘."

파이썬의 보물창고: 표준 라이브러리

설치 없이 바로 쓸 수 있는 도구들

- math: 수학 연산 (sqrt, pi)
- datetime: 날짜와 시간 (datetime.now())
- os / sys: 파일 시스템, 시스템 제어
- json: JSON 데이터 처리
- random: 난수 생성

```
import random
import datetime

print(random.randint(1, 45))  # random lottery-like number
print(datetime.datetime.now())
```

외부 라이브러리와 pip

표준 라이브러리에 없다면? 다운로드 받자!

- PyPI (Python Package Index): 전 세계 개발자들이 올린 패키지 저장소
- **설치 명령어 (터미널):**
  - `pip install pandas` (데이터 분석)
  - `pip install requests` (웹 요청)
  - `pip install openai` (GPT API)
- **가상 환경 (Virtual Environment):** 프로젝트별로 도구함을 따로 관리 (`venv`, `conda`)

모듈 똑똑하게 가져다 쓰기 (Import Styles)

- 1. **전체 가져오기**: `import pandas` → `pandas.read_csv()`
- 2. **줄여서 가져오기**: `import pandas as pd` → `pd.read_csv()`
- 3. **일부만 가져오기**: `from math import sqrt` → `sqrt(4)`

AI Prompt Tip
"주로 pd, np 같은 standard alias를 사용해서 짜줘."

메인 가드: `if __name__ == "__main__":`

Q. `import` 만 했을 뿐인데 코드가 실행돼요!

- 파이썬은 파일을 불러올 때 모든 코드를 실행해버립니다.
- **해결**: "이 파일이 직접 실행될 때만 작동해라"고 안전장치를 걸어야 합니다.

```python
def job():
    print("Doing work...")

# Without this block, job() runs on import.
if __name__ == "__main__":
    job()
```

Prompt: "모듈로 쓸 수 있게 메인 가드를 추가해줘."

데이터 구조화: 정보의 그릇 만들기

AI가 일을 잘하게 하려면 **"데이터의 형태(Format)"**를 정해줘야 합니다.

[Prompt]

"뉴스 기사를 담을 *JSON* 데이터 포맷을 설계해줘. 제목, 링크, 요약문, 발행일이 포함되어야 해."

```
news_item = {
    "title": "Future of AI",
    "link": "https://example.com/news/1",
    "summary": "AI is changing software development...",
    "published_at": "2026-02-11"
}
```

디버깅: AI에게 묻는 법

에러가 났을 때 당황하지 마세요. 에러 메시지는 **AI에게 주는 힌트**입니다.

AI 디버깅 3단계 루틴

1. **에러 복사**: 터미널의 빨간색 에러 메시지를 통째로 복사합니다.
2. **상황 설명**: "이 코드를 실행했는데 이런 에러가 났어."
3. **솔루션 요청**: "원인이 뭐고, 어떻게 고치면 될까?"

예외 처리 패턴: try-except-else-finally

- try: 에러가 날 수 있는 코드
- except: 에러 발생 시 대처
- else: 에러가 안 났을 때 실행
- finally: 무조건 실행 (정리 작업)

```
try:
    file = open('news.txt', 'r')
    content = file.read()
except FileNotFoundError:
    print("File not found.")
else:
    print(f"Read {len(content)} characters")
finally:
    print("Done.")  # always runs
```

Best Practice

파일을 다룰 때는 `with open(...) as f:` 패턴을 쓰면 finally로 닫을 필요가

자주 만나는 에러와 대처법

- `IndexError`: 리스트 범위를 벗어남 (`list[100]`)
- `KeyError`: 딕셔너리에 키가 없음 (`dict['unknown']`)
- `TypeError`: 사과와 오렌지를 더하려 함 (`1 + "1"`)
- `ValueError`: 값의 형식이 맞지 않음 (`int("abc")`)

Tip
에러 이름을 알면 `except IndexError:` 처럼 핀셋 처리가 가능합니다.

적극적 예외 발생: raise

Goal: 이상한 데이터가 들어오면 **일찍 멈추는 것**이 안전합니다.

```
def deposit(amount):
    if amount < 0:
        # intentionally raise an error to stop execution
        raise ValueError("amount must be greater than 0")

    print(f"{amount} deposited")
```

Prompt: "입력값이 유효하지 않으면 `ValueError`를 raise하도록 검증 로직을 추가해줘."

세션 6-8: 미니 프로젝트

실전 프로젝트: 나만의 뉴스 브리핑 봇

목표: 관심 있는 키워드의 뉴스를 자동으로 수집해서 요약 보여주기

시스템 파이프라인 (System Pipeline)

$$\text{RSS Feed (인터넷)} \xrightarrow{\text{수집}} \text{Python List (메모리)} \xrightarrow{\text{필터링}} \text{결과 (화면/파일)}$$

RSS와 XML: 배달부와 창고

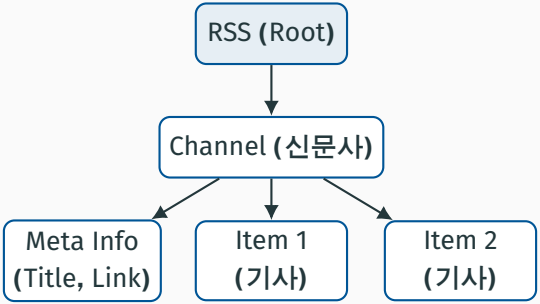
1. RSS (Really Simple Syndication)
   - 매일 사이트에 들어가지 않아도, **새 글이 올라오면 배달**해주는 서비스
   - "구독(Subscribe)" 버튼의 원조

2. XML (Extensible Markup Language)
   - HTML과 비슷하게 생겼지만, **목적**이 다름
   - HTML: 데이터를 **보여주기** 위함 (Display)
   - XML: 데이터를 **저장/전송**하기 위함 (Data)

비유

HTML은 신문의 '지면 레이아웃'이고, XML은 기사 내용이 담긴 '원고 파일'입니다.

RSS의 구조 (Tree Diagram)



Channel은 신문사 전체를, **Item**은 각각의 기사를 의미합니다.

실제 포맷 뜯어보기 (연합뉴스 예시)

```xml
<rss version="2.0">
  <channel>
    <title>연합뉴스 속보</title>
    <link>http://www.yonhapnews.co.kr</link>
    <description>실시간 뉴스 속보입니다.</description>

    <item>
      <title>가AI 바꾸는 미래 교육 현장</title>
      <link>http://yna.kr/view/AKR2026...</link>
      <description>인공지능 도입으로 교실이 변화하고 있습니다...</description>
      <pubDate>Tue, 11 Feb 2026 14:00:00 +0900</pubDate>
    </item>

    <item>
      <title>파이썬 개발자 수요 급증</title>
      <!-- ... -->
    </item>
  </channel>
</rss>
```

Step 1: 수집기(Fetcher) - `fetcher.py`

Goal: `feedparser`로 RSS 데이터를 가져오되, 차단을 피하고 에러를 처리한다.

Prompt for AI (Copilot / ChatGPT)
"파이썬 함수 `fetch_feed(url)`을 작성해줘.

1. **라이브러리**: `requests`와 `feedparser`를 사용해.
2. **차단 회피**: `headers` 딕셔너리에 'User-Agent'를 브라우저처럼 설정해서 `requests.get`에 넣어줘.
3. **에러 처리**: `try-except` 구문으로 네트워크 에러(`requests.exceptions.RequestException`)를 잡아서 에러 메시지를 출력하고 `None`을 반환해.
4. **반환값**: 파싱에 성공하면 `feed.entries` 리스트를 반환해."

Step 2: 데이터 가공(Parser) - `parser.py`

Goal: 정제되지 않은 HTML 태그를 제거하고, 날짜 형식을 통일한다.

Prompt for AI
"RSS 데이터를 정제하는 모듈을 만들거야. 다음 두 함수를 작성해.

1. `clean_html(text)`: `BeautifulSoup`을 사용해서 HTML 태그를 모두 제거하고 텍스트만 남겨.
2. `parse_feeds(entries)`: 피드 리스트를 받아서 순회해.
   - 각 글의 `title`, `link`, `summary`(태그 제거됨), `published`(YYYY-MM-DD 형식으로 변환) 정보를 담은 딕셔너리 리스트를 반환해.

`dateutil` 라이브러리를 써서 날짜 파싱을 처리해줘."

Step 3: 저장(Storage) - `storage.py`

Goal: 리스트 데이터를 JSON 파일로 영구 저장한다.

Prompt for AI
"데이터를 저장하는 `save_to_json(news_list, filename)` 함수를 작성해줘.

- `json` 모듈을 사용해서 리스트를 파일로 덤프(dump)해.
- **한글 깨짐 방지**: `ensure_ascii=False` 옵션을 반드시 켜줘.
- **인코딩**: 파일을 열 때 `encoding='utf-8'`을 명시해줘.
- `indent=4`를 줘서 보기 좋게 저장해."

Step 4: 시각화(HTML) - `html_generator.py`

Goal: 터미널이 아니라 예쁜 웹 페이지로 결과를 보자.

Prompt for AI
"뉴스 리스트를 입력받아 `news_dashboard.html` 파일을 생성하는 `generate_dashboard` 함수를 만들어줘.

1. **HTML 템플릿**: `f-string`을 사용해서 HTML 코드를 문자열로 작성해.
2. **스타일**: 내부 CSS(`<style>`)를 사용해서 카드(Card) 레이아웃으로 디자인해줘. (반응형 그리드, 쉐도우 효과, 마우스 호버 효과 포함)
3. 각 뉴스 카드에는 출처, 날짜, 제목, 요약, 'Read More' 버튼을 배치해줘."

Step 5: 메인 파이프라인(Main) - `main.py`

Goal: 모든 부품을 조립해서 작동시킨다.

Prompt for AI
"이제 만든 모듈들을 조립해서 실행하는 `main.py`를 작성해줘.

1. **URL 리스트**: 구글 뉴스 RSS URL 3개를 리스트로 정의해.
2. **실행 로직**:
   - 각 URL을 순회하며 `fetcher.fetch_feed`로 데이터를 가져와.
   - `parser.parse_feeds`로 데이터를 정제해.
   - `time.sleep(1)`을 줘서 서버 부하를 줄여.
3. **중복 제거**: 링크(Link)를 기준으로 중복된 기사를 제거해.
4. **결과 저장**: `storage.save_to_json`으로 저장하고, `html_generator.generate_dashboard`로 HTML을 생성해."

Step 6: 자동화(Automation) - 비서 채용

Goal: 내가 시키지 않아도 매일 아침 작동하기

Prompt for AI
"`schedule` 라이브러리를 사용해서, 위에서 작성한 `job()` 함수를 매일 아침 08:30에 자동으로 실행하게 해줘. 프로그램이 종료되지 않고 계속 대기(`while True`)해야 해."

- 이제 여러분의 컴퓨터만 켜져 있으면, AI 비서가 매일 아침 뉴스를 배달합니다.

오늘의 핵심 정리

1. **함수**로 AI에게 명확한 업무 지시서를 쓰는 법
2. **클래스**로 데이터와 기능을 묶어 관리하는 법
3. **모듈/패키지**로 코드를 레고 블록처럼 조립하는 법
4. **예외 처리**로 안전한 코드를 만드는 법
5. **프로젝트 설계**: AI에게 시스템 단위로 일을 시키는 법

마무리: AI 시대의 개발자

- Syntax(**문법**)는 AI가 더 잘 압니다.
- Context(**맥락**)와 Structure(**구조**)는 사람만이 설계할 수 있습니다.
- 오늘 배운 **구조화된 사고**와 **프롬프트 기술**로 여러분만의 시스템을 만들어보세요.

Q & A